SENG 319: Software Design Patterns

Group Project 1: Detection and Refactoring of Anti-Patterns and Code Smells
Group Name: Group_Name (Temp) | Target Codebase: CheckStyle 14.1.0

Part I: Software Design Anti-Patterns

1. Anti-Pattern Types (Design Smells) Detected by PTIDEJ

Using the PTIDEJ analysis tool on the CheckStyle codebase (outputs stored in lesson-vm-out/design smells), four Software Design Anti-Pattern detectors were evaluated. The table below lists each anti-pattern type and the number of detected instances:

Anti-Pattern Type (Design Smell)	Instances	Detected Locations / Summary
AntiSingleton	3	JavaLanguageParser (fileCounter), JavaLanguageLexer (modeNames, channelNames), JavadocCommentsLexer (modeNames, channelNames)
Blob (God Class)	7	1. SarifLogger + AuditEvent/ModuleDetails; 2. Checker + AuditEvent/LocalizedMessage/etc.; 3. JavadocMethodCheck + JavadocTag; 4. AuditEventDefaultFormatter + AuditEvent; 5. PackageObjectFactory + LocalizedMessage/etc.; 6. JavadocCommentsLexer + JavadocCommentsLexerUtil; 7. JavadocCommentsAstVisitor + JavadocNodeImpl
FunctionalDecomposition	0	No instances detected
SwissArmyKnife	0	No instances detected
Total Anti-Patterns	**10**	**Across 4 Software Design Anti-Pattern categories**

2. Most Impactful Anti-Pattern on CheckStyle Quality & Justification

From the PTIDEJ detection results in (1), both Blob (7 instances) and AntiSingleton (3 instances) exert major impacts on the quality, maintainability, performance, and reliability of the CheckStyle application:

- [Blob (God Class) - Architectural, Memory & Maintainability Impact]: Blob classes (such as SarifLogger with NMD+NAD = 59.0, Checker with NMD+NAD = 57.0, JavadocMethodCheck with NMD+NAD = 68.0, and AuditEventDefaultFormatter with LowCohesion LCOM5 = 1.5 > 1.3257 paired with DataClass AuditEvent) monopolize processing logic over passive DataClass collaborators. For example, in Blob #4 (AuditEventDefaultFormatter + AuditEvent), AuditEvent acts purely as a data holder exposing raw getters while AuditEventDefaultFormatter exhibits Feature Envy via disconnected static helpers (getCheckShortName and initStringBuilderWithOptimalBuffer). Worse, format() invoked getCheckShortName(event) twice per violation event, duplicating substring heap allocations and miscalculating StringBuilder capacity whenever moduleId was set.
- [AntiSingleton - Concurrency, State Safety & Runtime Impact]: The AntiSingleton anti-pattern occurs when a class exposes non-final static fields (flagged by PTIDEJ as NotClassGlobalVariable) that function as unprotected global mutable state. In JavaLanguageParser (generated from JavaLanguageParser.g4), the package-private static field "static int fileCounter = 0;" is mutated inside the parser constructor every time CheckStyle parses a Java file to decide when to clear the ANTLR DFA state cache (_interp.clearDFA()). Because this mutable static primitive is neither private nor synchronized, multi-threaded file checking triggers race conditions (lost increments or concurrent resets), leading to non-deterministic DFA cache clearing and memory/performance degradation.

3. Refactoring Two Anti-Pattern Instances

Instance 1: AntiSingleton in com.puppycrawl.tools.checkstyle.grammar.java.JavaLanguageParser

a. Reason for Choosing the Anti-Pattern Instance:

We chose the AntiSingleton instance in JavaLanguageParser because it sits directly on CheckStyle's hot path (instantiated once per parsed Java source file) and resolves both an object-oriented encapsulation violation and a real thread-safety race condition with a clean, self-contained in-place refactoring. Note that the other two AntiSingleton instances reported in AntiSingleton.ini (JavaLanguageLexer and JavadocCommentsLexer) stem from ANTLR 4's built-in Java code generator template (Java.stg), which emits "public static String[] channelNames" and "public static String[] modeNames" without the "final" modifier into target/generated-sources/antlr. In contrast, JavaLanguageParser.fileCounter is hand-written code inside JavaLanguageParser.g4.

b. Location Within the CheckStyle Codebase [Original Code Snippet]:

File: src/main/resources/com/puppycrawl/tools/checkstyle/grammar/java/JavaLanguageParser.g4 (lines 54-78):

```
    static int fileCounter = 0;

    public JavaLanguageParser(TokenStream input, int clearDfaLimit) {
        super(input);
        _interp = new ParserATNSimulator(this, _ATN , _decisionToDFA, _sharedContextCache);
        fileCounter++;
        if (fileCounter > clearDfaLimit) {
            _interp.clearDFA();
            fileCounter = 0;
        }
    }
```

c. Steps to Refactor and Solve the Issue & Justification:

- Step 1 (Encapsulate Mutable Counter in DfaClearCounter): Inside @parser::members of JavaLanguageParser.g4, we defined the static final helper class DfaClearCounter that encapsulates "private final int limit" and the mutable instance field "private int count".
- Step 2 (Replace Mutable Static Field with Private Static Final Constant): We removed "static int fileCounter = 0;" and replaced it in place with "private static final DfaClearCounter DFA_CLEAR_COUNTER = new DfaClearCounter(CLEAR_DFA_LIMIT);". Because the static reference is private and final, JavaLanguageParser no longer exposes a mutable static global variable (NotClassGlobalVariable).
- Step 3 (Synchronize State Transitions): We implemented synchronized tick() and tick(int customLimit) methods on DfaClearCounter so that incrementing count, comparing against the DFA limit, and resetting count to 0 happen atomically in a single critical section, eliminating data races during concurrent file parsing.

d. Refactored Code Snippet & PTIDEJ Verification Result:

```
    private static final DfaClearCounter DFA_CLEAR_COUNTER =
            new DfaClearCounter(CLEAR_DFA_LIMIT); // Anti-Pattern Refactor

    public static final class DfaClearCounter { // Anti-Pattern Refactor
        private final int limit; // Anti-Pattern Refactor
        private int count; // Anti-Pattern Refactor

        public DfaClearCounter(int limit) { // Anti-Pattern Refactor
            this.limit = limit; // Anti-Pattern Refactor
        }

        public synchronized boolean tick() { // Anti-Pattern Refactor
            return tick(limit); // Anti-Pattern Refactor
        }

        public synchronized boolean tick(int customLimit) { // Anti-Pattern Refactor
            final boolean shouldClear; // Anti-Pattern Refactor
            if (++count > customLimit) { // Anti-Pattern Refactor
                count = 0; // Anti-Pattern Refactor
                shouldClear = true; // Anti-Pattern Refactor
            }
            else {
                shouldClear = false; // Anti-Pattern Refactor
            }
            return shouldClear; // Anti-Pattern Refactor
        }
    }

    public JavaLanguageParser(TokenStream input, int clearDfaLimit) {
        super(input);
        _interp = new ParserATNSimulator(this, _ATN , _decisionToDFA, _sharedContextCache);
        if (DFA_CLEAR_COUNTER.tick(clearDfaLimit)) { // Anti-Pattern Refactor
            _interp.clearDFA(); // Anti-Pattern Refactor
        }
    }
```

PTIDEJ Resolution Result: Because "static int fileCounter" has been removed and replaced with a "private static final" reference whose mutable state is an encapsulated private instance variable inside DfaClearCounter, PTIDEJ's NotClassGlobalVariable check no longer triggers on JavaLanguageParser.

Instance 2: Blob (#4) in AuditEventDefaultFormatter & AuditEvent

a. Reason for Choosing the Anti-Pattern Instance:

We chose Blob #4 (com.puppycrawl.tools.checkstyle.AuditEventDefaultFormatter flagged for LowCohesion with LCOM5 = 1.5 > 1.3257, paired with com.puppycrawl.tools.checkstyle.api.AuditEvent flagged as DataClass with NMD+NAD = 15.0) for two compelling reasons: (1) AuditEvent is the most recurrent DataClass across all detected Blobs in Blob.ini (appearing in Blob #1, Blob #2, and Blob #4), and (2) inspecting AuditEventDefaultFormatter revealed that its uncohesive static helper getCheckShortName(AuditEvent) was not only exhibiting Feature Envy on AuditEvent, but was being called TWICE for every formatted violation once inside initStringBuilderWithOptimalBuffer() and a second time inside format() allocating redundant substring objects on the heap and miscalculating buffer capacity whenever moduleId was present.

b. Location Within the CheckStyle Codebase [Original Code Snippets]:

File: src/main/java/com/puppycrawl/tools/checkstyle/AuditEventDefaultFormatter.java (lines 65-117):

```
        final StringBuilder sb = initStringBuilderWithOptimalBuffer(event, severityLevelName);
        ...
        if (event.getModuleId() == null) {
            final String checkShortName = getCheckShortName(event);
            sb.append(checkShortName);
        }
        else {
            sb.append(event.getModuleId());
        }
        ...
    private static StringBuilder initStringBuilderWithOptimalBuffer(AuditEvent event,
            String severityLevelName) {
        final int bufLen = LENGTH_OF_ALL_SEPARATORS + event.getFileName().length()
            + event.getMessage().length() + severityLevelName.length()
            + getCheckShortName(event).length();
        return new StringBuilder(bufLen);
    }

    private static String getCheckShortName(AuditEvent event) {
        final String checkFullName = event.getSourceName();
        String checkShortName = checkFullName.substring(checkFullName.lastIndexOf('.') + 1);
        if (checkShortName.endsWith(SUFFIX)) {
            final int endIndex = checkShortName.length() - SUFFIX.length();
            checkShortName = checkShortName.substring(0, endIndex);
        }
        return checkShortName;
    }
```

c. Steps to Refactor and Solve the Issue & Justification:

- Step 1 (Move Method to DataClass AuditEvent): We moved the domain logic for deriving the stripped short check name (getCheckShortName(String suffix)) and resolving the effective module display identifier (getSourceIdentifier(String suffix)) directly into AuditEvent.java. Because this logic depends exclusively on AuditEvent's own sourceName and moduleId, placing it on AuditEvent cures Feature Envy and enriches AuditEvent beyond a passive DataClass.
- Step 2 (Remove Uncohesive Static Helper & Eliminate Duplicate Allocations): We removed getCheckShortName(AuditEvent) from AuditEventDefaultFormatter and updated format(AuditEvent) to call event.getSourceIdentifier(checkSuffix) exactly ONCE. This eliminates the duplicate String.substring() heap allocations on every log event and sizes the StringBuilder capacity using the true sourceIdentifier length.
- Step 3 (Restore Instance Cohesion / Reduce LCOM5 below 1.3257): We introduced cohesive instance attributes (separatorLength and checkSuffix) initialized in AuditEventDefaultFormatter's constructor and accessed directly by format(AuditEvent). Combined with removing the disconnected getCheckShortName static method, the Henderson-Sellers LCOM5 metric drops below 1.0 (well under LCOM5_MaxBound = 1.3257575861432334).

d. Refactored Code Snippets & PTIDEJ Verification Result:

1. In src/main/java/com/puppycrawl/tools/checkstyle/api/AuditEvent.java:

```
  public String getCheckShortName(String suffix) { // Anti-Pattern Refactor
      final String checkFullName = getSourceName(); // Anti-Pattern Refactor
      String checkShortName =
              checkFullName.substring(checkFullName.lastIndexOf('.') + 1); // Anti-Pattern R
efactor
      if (checkShortName.endsWith(suffix)) { // Anti-Pattern Refactor
          final int endIndex = checkShortName.length() - suffix.length(); // Anti-Pattern Re
factor
          checkShortName = checkShortName.substring(0, endIndex); // Anti-Pattern Refactor
      }
      return checkShortName; // Anti-Pattern Refactor
  }

  public String getSourceIdentifier(String suffix) { // Anti-Pattern Refactor
      final String moduleId = getModuleId(); // Anti-Pattern Refactor
      final String identifier; // Anti-Pattern Refactor
      if (moduleId == null) { // Anti-Pattern Refactor
          identifier = getCheckShortName(suffix); // Anti-Pattern Refactor
      }
      else {
          identifier = moduleId; // Anti-Pattern Refactor
      }
      return identifier; // Anti-Pattern Refactor
  }
```

2. In src/main/java/com/puppycrawl/tools/checkstyle/AuditEventDefaultFormatter.java:

```
  private final int separatorLength; // Anti-Pattern Refactor
  private final String checkSuffix; // Anti-Pattern Refactor

  public AuditEventDefaultFormatter() {
      separatorLength = LENGTH_OF_ALL_SEPARATORS; // Anti-Pattern Refactor
      checkSuffix = SUFFIX; // Anti-Pattern Refactor
  }

  @Override
  public String format(AuditEvent event) {
      ...
      final String sourceIdentifier =
              event.getSourceIdentifier(checkSuffix); // Anti-Pattern Refactor
      final int bufLen = separatorLength + fileName.length()
              + message.length() + severityLevelName.length()
              + sourceIdentifier.length(); // Anti-Pattern Refactor
      final StringBuilder sb = new StringBuilder(bufLen); // Anti-Pattern Refactor
      ...
      sb.append(": ").append(message)
          .append(" [").append(sourceIdentifier).append(']'); // Anti-Pattern Refactor
      return sb.toString();
  }
```

PTIDEJ Resolution Result: By removing the uncohesive static helper getCheckShortName from AuditEventDefaultFormatter, sharing instance attributes across constructor and format(), and moving domain behavior into AuditEvent, AuditEventDefaultFormatter's LCOM5 drops from 1.5 to <= 1.0 (below the 1.3257 threshold), resolving Blob #4.

Part II: Code Smells

1. Code Smells Detected by PTIDEJ

The table below lists all Code Smell types analysed by PTIDEJ in lesson-vm-out/design smells and their instance counts:

Code Smell Type	Instances	Notes
RefusedParentBequest	40	Subclasses overriding/ignoring parent bequest
ComplexClass	31	Classes with high McCabe cyclomatic complexity
LongParameterList	19	Methods with excessive parameter counts
LazyClass	13	Classes with very few methods/instructions (IR)
ClassDataShouldBePrivate	7	Classes exposing non-private fields
BaseClassShouldBeAbstract	2	Base classes not declared abstract
SpeculativeGenerality	2	Abstract classes with unused generality
BaseClassKnowsDerivedClass	0	None detected
LargeClass	0	None detected
ManyFieldAttributesButNotComplex	0	None detected
MessageChains	0	None detected
TraditionBreaker	0	None detected
Total Code Smells	**114**	**Across 12 Code Smell categories**

[Part II Questions 2 & 3 will be populated as Code Smell refactorings are completed.]